

Bakalářské zkoušky (příklady otázek)

2025-06-23

1 Deterministické konečné automaty (společné okruhy)

1. Uveďte definici jazyka $L(A)$ přijímaného daným deterministickým konečným automatem $A = (Q, \Sigma, \delta, q_0, F)$ včetně definice jeho rozšířené přechodové funkce δ^* .
2. Nechť $S \subseteq \Sigma^*$ je neprázdná konečná množina slov, Q je množina všech prefixů slov z S (včetně prázdného slova ε a slov z S) a $A = (Q, \Sigma, \delta, \varepsilon, S)$ je deterministický konečný automat s množinou stavů Q . Definujte jeho přechodovou funkci δ tak, aby přijímal jazyk

$$L(A) = \{w \in \Sigma^* \mid w \text{ obsahuje nějaké } s \in S \text{ (jako podslovo)}\}.$$

3. Sestrojte deterministický konečný automat přijímající jazyk

$$\{w \in \{0, 1\}^* \mid w \text{ obsahuje } 010 \text{ nebo } \underline{\text{končí}} \text{ na } 10\}.$$

Nástin řešení

1. $L(A) := \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\}$, kde $\delta^*: Q \times \Sigma^* \rightarrow Q$ je definována induktivně

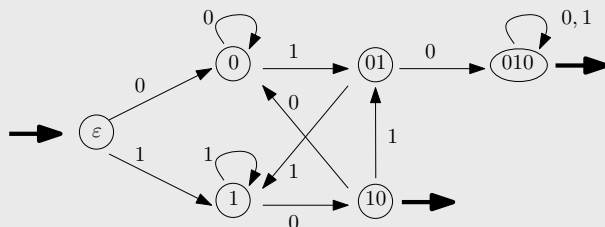
$$\begin{aligned}\delta^*(q, \varepsilon) &:= q, \\ \delta^*(q, wx) &:= \delta(\delta^*(q, w), x)\end{aligned}$$

pro každé $q \in Q$, $w \in \Sigma^*$ a $x \in \Sigma$.

2. Pro každé $w \in Q$ a $x \in \Sigma$,

$$\delta(w, x) := \begin{cases} w, & \text{pokud } w \in S, \\ s, & \text{pokud } w \notin S \text{ a } wx \text{ končí na nějaké } s \in S \text{ (vezmi např. nejkratší),} \\ v, & \text{kde } v \text{ je nejdelší sufix slova } wx, \text{ který je v } Q, \text{ v ostatních případech.} \end{cases}$$

3. Automat $A = (\{\varepsilon, 0, 1, 01, 10, 010\}, \{0, 1\}, \delta, \varepsilon, \{10, 010\})$ je na obrázku níže.



2 Jednoduchý správce paměti (společné okruhy)

Předpokládejte v celé otázce pouze kontext 32-bitového *little endian* procesoru, *bítem 0 rozumíme LSB*.

Naším cílem je v souvislém bloku paměti implementovat jednoduchý heap pro dynamickou alokaci podbloků paměti (neplést s haldou jako datovou strukturou). Celý náš heap bude mít pevnou velikost danou při inicializaci a nebude používat žádné

optimalizace nad rámec zde popsaného. Paměť našeho heapu si chceme organizovat jako souvislou posloupnost podbloků, kde každý podblok má být vždy **zarovnaný na 2 byty**. Každý podblok heapu je buď *volný* nebo *obsazený* (naalokovaný).

Volné podbloky mají následující strukturu:

+0: **Size** [16-bit] = velikost podbloku **včetně této hlavičky**. Jelikož z výše uvedeného vyplývá, že bit 0 této položky by byl vždy 0, tak tuto nulu neukládáme. Místo toho pro nás bude mít bit 0 položky **Size** speciální význam – bude označovat, zda je podblok volný nebo obsazený. 0 = *volný*, 1 = *obsazený*. Tedy pro volný podblok:

bit 0: 0 = volný

+2: **Next** [16-bit] = offset dalšího volného podbloku od začátku celého heapu – tedy volné podbloky tvoří vlastně jednosměrně vázaný seznam. Poslední volný podblok má v této položce vyplněnou hodnotu -1.

+4: zbytek podbloku je nevyužitý a je vyplněn nulovými byty.

Obsazené podbloky mají následující strukturu:

+0: **Size** [16-bit] = má stejný význam jako u volného podbloku, s tím rozdílem, že zde:

bit 0: 1 = obsazený

+2: **Payload** – samotná data podbloku (naš alokátor vrací všechny tyto byty vynulované).

Pozor: obsazené podbloky tedy nemají položku **Next** – místo v paměti, které u volného podbloku náleželo položce **Next**, se alokací podbloku stane součástí **Payloadu** a musí být alokatorem na haldě vynulováno.

Pro celý heap si pamatujeme offset prvního volného podbloku od začátku celého heapu. Na počátku se heap skládá právě z 1 volného podbloku, jehož velikost odpovídá velikosti celého heapu. Při požadavku na alokaci nového podbloku dostaneme požadovanou velikost payloadu v bytech a použijeme první dostatečně velký volný podblok na nejnižším offsetu od začátku heapu (tj. strategie first-fit).

Předpokládejte následující kostru třídy v C#, kterou budete doplňovat (požadovanou implementaci můžete napsat v jazyce C# nebo Java nebo C++ – pro Javu nebo C++ předpokládejte ekvivalentní implementaci kostry v daném jazyce s tím, že parametry konstruktoru a metod si můžete vhodně upravit dle zvyklostí v daném jazyce) (**ushort** je 16-bitové bezznaménkové celé číslo):

```
class Heap {
    private ushort _firstFreeOffset = 0;
    private byte[] _heap;

    public Heap(byte[] availableMemory) {
        _heap = availableMemory;
        // TODO
    }
    private ushort FindFirstFree(ushort payloadSize) { // TODO }
    private void Mark(ushort offset, bool isFree) { // TODO }

    public void SetUshort(ushort offset, ushort value) { ... }
    public ushort GetUshort(ushort offset) { ... }
    public ushort Alloc(ushort payloadSize) { ... }
    public void Free(ushort payloadOffset) { ... }
}
```

Metody **SetUshort** a **GetUshort** neprogramujte, ale naopak je vhodně využijte ve svém kódu. Metoda **SetUshort** na **offset** od začátku heapu uloží **value** v little endian pořadí. Metoda **GetUshort** vrátí 16-bitovou hodnotu uloženou na heapu od daného **offsetu** (byty na heapu chápe v little endian pořadí).

Metody **Alloc** a **Free** neprogramujte – tyto metody bude programovat někdo jiný s využitím vašich metod. Pro úkol 4 níže se hodí vědět: metoda **Alloc** naalokuje na heapu místo pro **payloadSize** velký payload a vrátí **offset** pro payload; metoda **Free** bere **offset** payloadu vráceného nějakým voláním metody **Alloc** a podblok, do kterého payload náleží, uvolní.

Úkoly:

1. Doplňte implementaci konstruktoru, který inicializuje prázdný heap.
2. Napište implementaci metody **FindFirstFree**, která vrátí **offset** prvního volného podbloku vhodné velikosti. Stav podbloku tato funkce nijak nemění.
3. Napište implementaci metody **Mark**, která očekává, že na **offsetu** je platný podblok haldy, a změní jeho stav na „obsazený“ nebo „volný“ dle parametru **isFree**. Funkce nebude manipulovat s položkou **Next**.
4. Předpokládejte, že jsme pro náš heap dostali k dispozici 22 bytů – tato paměť je předem vyplněna nulovými byty. Předpokládejte, že provedeme s heapem následující operace:

```

A = Alloc(2)
B = Alloc(4)
C = Alloc(2)
Free(B)

```

Napište hexdump 22 bytů paměti konečného stavu heapu po provedení všech těchto operací – řešení vyplňte do jedné z níže uvedených tabulek (je jich zde redundantně více, kdybyste své řešení potřebovali opravit).

offset	+0	+1	+2	+3	+4	+5	+6	+7
0x0000								
0x0008								
0x0010								

offset	+0	+1	+2	+3	+4	+5	+6	+7
0x0000								
0x0008								
0x0010								

Nástin řešení

offset	+0	+1	+2	+3	+4	+5	+6	+7
0x0000	05	00	00	00	06	00	0E	00
0x0008	00	00	05	00	00	00	08	00
0x0010	FF	FF	00	00	00	00	–	–

3 Skalární součin (společné okruhy)

Uvažujme zobrazení ve tvaru $\langle x, y \rangle = x^T A y$ pro symetrickou matici $A \in \mathbb{R}^{n \times n}$.

1. Rozhodněte o platnosti výroků:

(a) Je-li $\langle x, y \rangle = x^T A y$ skalární součin na $\mathbb{R}^n$, pak $\det(A) > 0$.

(b) Je-li $B \in \mathbb{R}^{n \times n}$ symetrická regulární matice, pak $\langle x, y \rangle = x^T B^2 y$ je skalární součin na $\mathbb{R}^n$.

2. Uvažujme skalární součin na $\mathbb{R}^n$ zadaný ve tvaru $\langle x, y \rangle = x^T A y$ pro nějakou vhodnou matici $A \in \mathbb{R}^{n \times n}$. Zapište Cauchyho–Schwarzovu nerovnost tak, aby místo normy a skalárního součinu byly použity maticové a aritmetické operace.

3. Rozhodněte, zda pro všechna přirozená $n \geq 1$ platí, že matice

$$A = \begin{pmatrix} 1 & \dots & \dots & 1 \\ \vdots & 2 & \dots & 2 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 2 & \dots & n \end{pmatrix}$$

(tj. $A \in \mathbb{R}^{n \times n}$ s prvky $a_{i,j} = \min(i, j)$) je pozitivně definitní. Formulujte příslušné kritérium a ověřte ho, případně ukažte protipříklad.

Nástin řešení

1. (a) Ano, matice je pozitivně definitní, má tedy kladná vlastní čísla i determinant.

(b) Ano, B^2 je symetrická s kladnými vlastními čísly, tedy pozitivně definitní.

2. Cauchyho–Schwarzova nerovnost: Pro každé $x, y \in V$ platí

$$|\langle x, y \rangle| \leq \|x\| \cdot \|y\|.$$

V našem případě má nerovnost tvar

$$|x^T A y| \leq \sqrt{x^T A x} \sqrt{y^T A y}.$$

3. Ano, matice A je pozitivně definitní. Například Choleského rozklad má tvar $A = LL^T$, kde

$$L = \begin{pmatrix} 1 & 0 & \dots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ \vdots & & \ddots & 0 \\ 1 & \dots & \dots & 1 \end{pmatrix}$$

Snadno lze ověřit i Gaussovou eliminací nebo Sylvestrovým kritériem.

4 Centrální limitní věta (společné okruhy)

1. Napište znění Centrální limitní věty.

2. Předpokládejme, že program jeden vstup zpracovává náhodnou dobu se střední hodnotou 100 ms a směrodatnou odchylkou 20 ms. Použijte Centrální limitní větu pro odhad pravděpodobnosti, že zpracování sto vstupů bude trvat více než 10.3 s. Napište přesnou formuli pomocí Φ a také vyčíslete pomocí tabulky níže.

3. Vysvětlíte, jaké jsme udělali předpoklady (záleží na tom, jaké je rozdělení doby běhu programu?) a jak jsou realistické.

x	-2.5	-2.0	-1.5	-1.0	-0.5	0.0	0.5	1.0	1.5	2.0	2.5
$\Phi(x)$	0.01	0.02	0.07	0.16	0.31	0.5	0.69	0.84	0.93	0.98	0.99

Nástin řešení 1. Znění věty: Necht' $X_1, X_2, \dots$ jsou stejně rozdělené n.n.v. se střední hodnotou μ a rozptylem σ^2 . Označme $Y_n = \frac{(X_1 + \dots + X_n) - n\mu}{\sqrt{n} \cdot \sigma}$.

Pak

$$Y_n \xrightarrow{d} N(0, 1).$$

Neboli, pokud F_n je distribuční funkce Y_n , tak

$$\lim_{n \rightarrow \infty} F_n(x) = \Phi(x) \quad \text{pro každé } x \in \mathbb{R}.$$

Říkáme, že posloupnost Y_n konverguje k $N(0, 1)$ v *distribuci*.

2. Označme X_i dobu běhu pro i -tý vstup. To, že $X_1 + \dots + X_{100} > 10.3$ je ekvivalentní tomu, že $Y_{100} > \frac{10.3 - 100 \cdot 0.1}{\sqrt{100} \cdot 0.02} = 1.5$. Podle CLV je $P(Y_{100} < 1.5) \approx \Phi(1.5)$. Proto hledaná pravděpodobnost je přibližně $1 - \Phi(1.5) \approx 1 - 0.93 = 0.07$, neboli zhruba 7 %.

3. CLV nepředpokládá nic speciálního o rozdělení jednotlivých X_i (pokud jsou veličiny stejně rozdělené s konečnou střední hodnotou a rozptylem). Předpokládáme ale nezávislost – to může být problém; pokud je například počítač přetížen, bude na všech vstupech pomalejší. Také nahrazujeme limitou hodnotu pro $n = 100$, to ale typicky při aplikaci CLV není problém.